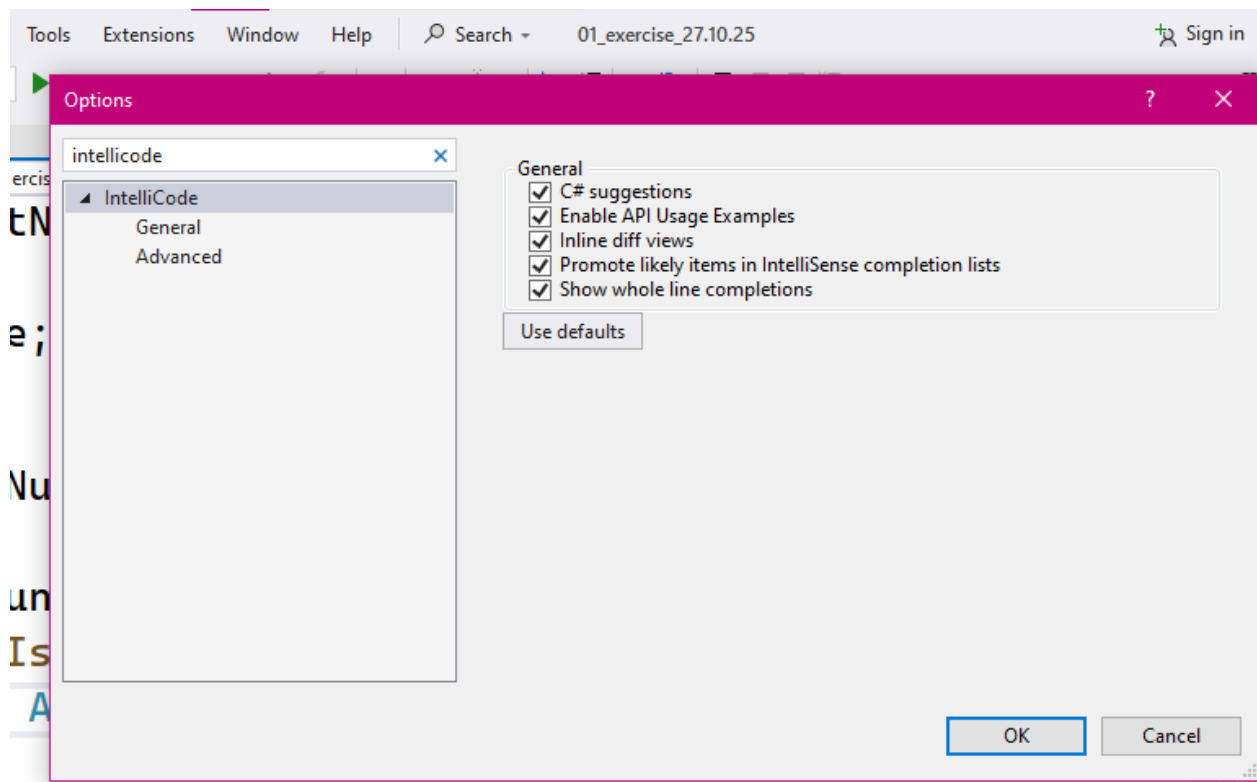


Уточнения върху конструктори:

Как да включите IntelliSense, ако е изключен: Tools -> Options



Допълнително =>

Операторът => в C# е **“lambda оператор”** (нарича се и **стрелков оператор**, от английски *lambda arrow*). Използва се за **създаване на ламбда изрази (lambda expressions)** — кратък начин да се дефинира **анонимна функция** (функция без име).

Както съм казвала и друг път, => е кратък запис, който сочи към обект, който взема и замества, например в **търсачка**, при условие .Where(), .Select()... и **вместо return**.

Language-Integrated Query (LINQ) – технологии за заявки в C#

```

class Rectangle
{
    public double Width { get; set; }
    public double Height { get; set; }

    // Изразно тяло на метод вместо return
    public double Area() => Width * Height;

    // Изразно тяло на свойство само за четене
    public double Perimeter => 2 * (Width + Height);
                                get;
}

```

Обърнете внимание, че е при read-only – когато имаме само get без set:

```

// ❌ Традиционен запис
public string FullName
{
    get { return $"{FirstName} {LastName}"; }
}

// ✅ Съкратен запис с =>
public string FullName => $"{FirstName} {LastName}";

```

Подходящ е за прости методи вместо return:

```

class Calculator
{
    // ✅ Методи с =>
    public int Add(int a, int b) => a + b;
    public int Multiply(int a, int b) => a * b;
    public bool IsEven(int number) => number % 2 == 0;
    public string FormatNumber(int num) => $"Number: {num}";

    // ❌ Не може за сложни методи
    // public void ComplexMethod() => { Console.WriteLine("Test"); } // ГРЕШКА!

    // ✅ Но може за void методи с една команда
    public void PrintHello() => Console.WriteLine("Hello!");
}

```

```

class CartItem
{
    public string Name { get; }
    public decimal Price { get; }
    public int Quantity { get; set; } = 1;

    public CartItem(string name, decimal price) => (Name, Price) = (name, price);
}

```

Ето един лесен нагледен пример за клас:

```

internal class PersonLambda
{
    private DateTime birthDate;
    0 references
    public DateTime BirthDate
    {
        get => birthDate;
        set => birthDate = value;
    }

    2 references
    public string FirstName { get; set; }
    2 references
    public string LastName { get; set; }

    // Read-only cBoйcmBa - Bмecmo get c =>
    0 references
    public string FullName => $"{FirstName} {LastName}";
    1 reference
    public int Age => DateTime.Now.Year - birthDate.Year;
    0 references
    public bool IsAdult => Age >= 18;

    0 references
    public PersonLambda(string firstName, string lastName, DateTime birthDate)
    {
        FirstName = firstName;
        LastName = lastName;
        this.birthDate = birthDate;
    }
}

```

Ако използваме само private, в конструктора използваме this. , но ако имаме валидация със set, заради нея използваме директно public, който така или иначе съдържа private + проверка.

```

internal class PersonLambda
{
    private DateTime birthDate;
    public DateTime BirthDate
    {
        get => birthDate;
        set => birthDate = value;
    }
}

```

```

    public string FirstName { get; set; }
    public string LastName { get; set; }

    // Read-only свойства - вместо get с =>
    public string FullName => $"{FirstName} {LastName}";
    public int Age => DateTime.Now.Year - birthDate.Year;
    public bool IsAdult => Age >= 18;

    public PersonLambda(string firstName, string lastName, DateTime birthDate)
    {
        FirstName = firstName;
        LastName = lastName;
        this.birthDate = birthDate;
    }
}

```

Подготовка за изпитване:

1 зад от файл 16.06 с 3 примерни задачи на г-жа Виолета Маринова. Другите 2 са подобни за упражнение и подготовка:

Задача 1: Клас Account (Банков акаунт)

◆ Описание:

Да се създаде клас **Account**, който представя банков акаунт на клиент.

◆ Полета:

- **accountNumber** – частно (private) поле от тип `string`, което съхранява номера на акаунта.
- **balance** – частно (private) поле от тип `double`, което съхранява текущия баланс по акаунта.

◆ Свойства:

- **AccountNumber** – публично (public) свойство, чрез което се достъпва и променя стойността на `accountNumber`.
 - Свойството трябва да извършва **валидация**:
 - Номерът на акаунта не може да бъде празен низ.
 - Номерът на акаунта трябва да съдържа **поне 5 символа**.
- **Balance** – публично (public) свойство, чрез което се достъпва и променя стойността на `balance`.
 - Свойството трябва да извършва **валидация**:
 - Балансът не може да бъде отрицателен (трябва да е ≥ 0).

◆ Конструктор:

Да се създаде **конструктор с параметри**, който приема:

- `accountNumber` (string)
- `balance` (double)

При създаването на обект чрез конструктора се задават началните стойности на свойствата, като се извършва горепосочената **валидация**.

Ако някоя стойност е невалидна, да се изведе съобщение за грешка (или да се хвърли изключение).

✦ Въвеждане на данни от конзолата:

- Да се въведат от клавиатурата номер на акаунта и начален баланс за **два различни акаунта**.
- Въвеждането става чрез командите `Console.ReadLine()`.

✦ Създаване на обекти:

- Да се създадат **два обекта** от класа `Account`, като се използва конструкторът с параметри и въведените от конзолата стойности.

✦ Извеждане:

- Да се изведе на екрана информация за всеки акаунт:
 - Номер на акаунта

Баланс (с две цифри след десетичната запетая)

Решение с коментари:

```
Account.cs* x User.cs Car.cs Person.cs Reverse.cs Recursions.cs Country.cs Overloads.cs
01_exercise_27_10_25.Account Balance

7 references
internal class Account
{
1   private string accountNumber;
    private double balance;

    2 references
    public string AccountNumber
    {
        get { return accountNumber; } // get => accountNumber;
        set { if (string.IsNullOrEmpty(value))
              throw new ArgumentException("Грешка: Номерът на акаунта не може да бъде празен!");
              if (value.Trim().Length < 5)
                  throw new ArgumentException("Грешка: Номерът на акаунта трябва да съдържа поне 5 символа!");
              без else
                  accountNumber = value.Trim();
        }
    }
    2 references
    public double Balance
    {
        get => balance; МОЖЕ И С НОВИЯ СИНТАКСИС
        set
        {
            if (value < 0)
                throw new ArgumentException("Грешка: Балансът не може да бъде отрицателен!");

            balance = Math.Round(value, 2);
        }
    }
}
```

В set може да има по няколко **само if**-конструкции. Няма голямо значение какъв точно **Exception** ще хвърлим засега, важното е, че след него, не се стига до присвояване на стойност, нищо, че **няма else**, просто грешката ще прекрати изпълнението. Също, вътре във валидациите на set, навсякъде предпочитаме ключовата дума **value**, а private-променливата е **последна** само за присвояването.

```
3 references
public Account(string accountNumber, double balance)
{
    AccountNumber = accountNumber; // Public, започва set-проверка
    Balance = balance;
}
```

```
3 references
public void PrintInfo() стандартно разпечатване
{
    Console.WriteLine($"Номер на акаунт: {AccountNumber}");
    Console.WriteLine($"Баланс: {Balance:F2} лв.");
    Console.WriteLine(new string('-', 30));
}
```

```
internal class Account
{
    private string accountNumber;
    private double balance;

    public string AccountNumber
```

```

    {
        get { return accountNumber; } // get => accountNumber;
        set { if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Грешка: Номерът на акаунта не може
да бъде празен!");
            if (value.Trim().Length < 5)
                throw new ArgumentException("Грешка: Номерът на акаунта трябва
да съдържа поне 5 символа!");

            accountNumber = value.Trim();
        }
    }
    public double Balance
    {
        get => balance;
        set
        {
            if (value < 0)
                throw new ArgumentException("Грешка: Балансът не може да бъде
отрицателен!");

            balance = Math.Round(value, 2);
        }
    }

    public Account(string accountNumber, double balance)
    {
        AccountNumber = accountNumber; // Public, заради set-проверката
        Balance = balance;
    }

    public void PrintInfo()
    {
        Console.WriteLine($"Номер на акаунт: {AccountNumber}");
        Console.WriteLine($"Баланс: {Balance:F2} лв.");
        Console.WriteLine(new string('-', 30));
    }
}

```

Няколко теста с 4 обекта и различни грешки в Main():

```

Account account1 = new Account("N12345", 230);
account1.PrintInfo();

//Account account2 = new Account(" ", 24);
//account2.PrintInfo();

Account account3 = new Account("1234", 34);
account3.PrintInfo();

Account account4 = new Account("w12356", -4);
account4.PrintInfo();

```

```
D:\Exercises\11 class\01_exercise_27.10.25\bin\Debug\net8.0\01_exercise_27.10.25.exe
Номер на акаунт: N12345 account1 - верен
Баланс: 230.00 лв.
-----
Unhandled exception. System.ArgumentException: грешка: Номерът на акаунта трябва да съдържа поне 5 символа!
   at 01_exercise_27.10.25.Account.set_AccountNumber(String value) in D:\Exercises\11 class\01_exercise_27.10.25\Account.cs:line 20
   at 01_exercise_27.10.25.Account..ctor(String accountNumber, Double balance) in D:\Exercises\11 class\01_exercise_27.10.25\Account.cs:line 39
   at 01_exercise_27.10.25.Program.Main(String[] args) in D:\Exercises\11 class\01_exercise_27.10.25\Program.cs:line 78
```

```
Account account1 = new Account("N12345", 230);
account1.PrintInfo();
```

```
//Account account2 = new Account(" ", 24);
//account2.PrintInfo();
```

```
Account account3 = new Account("1234", 34);
account3.PrintInfo();
```

```
Account account4 = new Account("w12356", -4);
account4.PrintInfo();
```